



Contents lists available at [SciVerse ScienceDirect](http://www.sciencedirect.com)

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss



Reconciling software development models: A quasi-systematic review

Andréa Magalhães Magdaleno^{a,b,*}, Cláudia Maria Lima Werner^a, Renata Mendes de Araujo^{b,c}

^a COPPE/UFRJ – Systems Engineering and Computer Science Program, Federal University of Rio de Janeiro (UFRJ), P.O. Box 68511, Rio de Janeiro 21945-970, RJ, Brazil

^b NP2Tec – Research and Practice Group in Information Technology, Brazil

^c Graduate Program in Informatics (PPGI), Federal University of Rio de Janeiro State (UNIRIO), Urca, Rio de Janeiro, RJ, Brazil

ARTICLE INFO

Article history:

Received 24 January 2010

Received in revised form 22 August 2011

Accepted 23 August 2011

Available online xxx

Keywords:

Systematic review

Software process

Reconciliation among development models

Plan-driven

Agile

Free/open source software

ABSTRACT

Purpose: The purpose of this paper is to characterize reconciliation among the plan-driven, agile, and free/open source software models of software development.

Design/methodology/approach: An automated quasi-systematic review identified 42 papers, which were then analyzed.

Findings: The main findings are: there exist distinct – organization, group and process – levels of reconciliation; few studies deal with reconciliation among the three models of development; a significant amount of work addresses reconciliation between plan-driven and agile development; several large organizations (such as Microsoft, Motorola, and Philips) are interested in trying to combine these models; and reconciliation among software development models is still an open issue, since it is an emerging area and research on most proposals is at an early stage.

Research limitations: Automated searches may not capture relevant papers in publications that are not indexed. Other data sources not amenable to execution of the protocol were not used. Data extraction was performed by only one researcher, which may increase the risk of threats to internal validity.

Implications: This characterization is important for practitioners wanting to be current with the state of research. This review will also assist the scientific community working with software development processes to build a common understanding of the challenges that must be faced, and to identify areas where research is lacking. Finally, the results will be useful to software industry that is calling for solutions in this area.

Originality/value: There is no other systematic review on this subject, and reconciliation among software development models is an emerging area. This study helps to identify and consolidate the work done so far and to guide future research. The conclusions are an important step towards expanding the body of knowledge in the field.

© 2011 Elsevier Inc. All rights reserved.

1. Introduction

Software development organizations are continually challenged by the need to improve the quality of software products. Technologies are changing rapidly and software is becoming larger and more complex. In addition, large-scale, distributed development poses new challenges.

In this context, the assumption that the software development process directly influences the quality of the product developed (Cugola and Ghezzi, 1998; Pressman, 2001; Raman, 2000) has

motivated many organizations to adopt maturity models, such as CMMI-DEV (Capability Maturity Model Integration) (Chrissis et al., 2006), or quality standards, like ISO 12207 (ISO/IEC, 1995). As a result, the plan-driven development model has become less chaotic, more predictable and more manageable.

The success of some free/open source software (FOSS) projects, such as Linux, Apache and Mozilla, also caught the attention of academia, industry and users, because of their capability to produce high-quality software quickly and for free (Cubranic and Booth, 1999; Feller and Fitzgerald, 2001; Raymond, 2001). The FOSS development model does not have a formal, disciplined definition, since it works based predominantly on voluntary collaboration.

Agile methods, using shorter development cycles and higher levels of client involvement and participation to cope with the changes that occur in the course of a development project, are often presented as an alternative to plan-driven software development (Dyba and Dingsoyr, 2008). Agile methods advocate

* Corresponding author at: COPPE/UFRJ – Systems Engineering and Computer Science Program, Federal University of Rio de Janeiro (UFRJ), P.O. Box 68511, Rio de Janeiro 21945-970, RJ, Brazil. Tel.: +55 21 2562 8702; fax: +55 21 2562 8676.

E-mail addresses: andrea@cos.ufrj.br (A.M. Magdaleno), werner@cos.ufrj.br (C.M.L. Werner), renata.araujo@uniriotec.br (R.M. Araujo).

software development with minimal documentation, based on self-organized teams that use the tacit knowledge acquired during the project (Cockburn, 2001).

Each of these three development models has its peculiarities, successes and challenges, and has taken separate paths. Due to differences in vocabulary, misinterpretation and misuse of approaches, a common view is that plan-driven and agile are poles apart (Boehm, 2002; Boehm and Turner, 2003b; Glass, 2001; Glazer et al., 2008; Santana et al., 2009). While plan-driven and FOSS also tend to be perceived as opponents, FOSS and agile tend to be perceived as more similar than different (Abrahamsson et al., 2002; Simmons and Dillon, 2003; Theunissen et al., 2008; Warsta and Abrahamsson, 2003). Both agile and FOSS emphasize iterative development and frequent releases, with minimum documentation and the focus on testing (Barnett, 2004). However, it is also important to note that there is a key difference between FOSS and agile: several agile methods (like Extreme Programming) emphasize co-location, while FOSS typically involves geographically distributed developers. Another contrast is that FOSS relies on tools to make its distributed approach work, while agile methods do not. Therefore, FOSS is similar to plan-driven in all the same respects that it differs from agile: both FOSS and plan-driven favor distributed development, as well as the use of tools to support development tasks.

In order to examine how the particularities of each development model might be accommodated, we conducted a quasi-systematic review of the technical literature with a view to characterizing reconciliation among software development models. Reconciliation is defined as the act of re-establishing normal relations between belligerents. In software development, it is an attempt to approximate the different development models in order to establish more effective software processes. Since there is no other systematic review on this topic and reconciliation is an emerging area, this study is useful to identify and consolidate work to date and to guide future research. The conclusions are an important step towards expanding the body of knowledge in the area.

The remainder of this paper is organized as follows. In Section 2, we present the main characteristics of each software development model. Section 3 describes the main objectives and phases of the systematic review process. Section 4 details the plan of this quasi-systematic review, defining a review protocol that specifies the main and secondary research questions. In Section 5, the execution of the protocol outlined in Section 4 is described. In Section 6, we report and analyze the results of this quasi-systematic review. In Section 7, the research questions are discussed and answered, characterizing reconciliation among software development models. Finally, Section 8 concludes the paper, discusses its limitations and indicates some opportunities for future work.

2. Software development models

A software development model defines, at a high-level of abstraction, a set of best practices for software development (Sommerville, 2004). These practices are organized into a software process that corresponds to the “set of Software Engineering tasks needed to transform user requirements into software” (Humphrey, 1989). Fuggetta (2000) defined a software process as “a coherent set of policies, organizational structures, technologies, procedures, and artifacts required to design, develop, deploy and maintain a software product”.

Plan-driven, agile and FOSS development models have the same goal: to improve the software development process, but they adopt different approaches. In the past decade, these three models have had an enormous impact on software industry and their future outlooks are also promising (Ebert, 2007; Theunissen et al., 2008).

Each model represents a distinct development universe, differing in philosophy and main characteristics. While the plan-driven model seeks predictability, stability and reliability (Chrissis et al., 2006), agile development tries quickly to add value to business and adapt to changes (Cockburn, 2001). Meanwhile, in FOSS development, the main objective is to guarantee user freedom (FSF, 2008).

The plan-driven model is typically exemplified by maturity models such as CMMI-DEV (Chrissis et al., 2006), or quality standards, like ISO 12207 (ISO/IEC, 1995). Historically, this development model was designed to handle high-risk environments and large and complex projects with high costs involved, where relationships between client and development team were characterized by low trust and were governed by contractual definitions (Glazer et al., 2008). Besides, due to this lack of trust, frequent approvals are necessary in each stage of the development lifecycle. There is also a concern with the completeness of documentation and the traceability between documentation generated in each phase. In this scenario, the plan-driven model is characterized by its orientation to detailed planning and emphasis on well-defined and continuous improved processes.

The assumptions of agile development, as observed in methods such as Extreme Programming (XP) (Beck, 2004), Scrum (Schwaber, 2004), Crystal (Cockburn, 2004), Kanban (Anderson, 2010) can be summarized by the four values of the Agile Manifesto (Beck et al., 2001): individuals and interactions over processes and tools; software running over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a plan. In general, the main objectives of agile methods are to deliver value quickly to customers and to be responsive to change in market, technology and environment (Cockburn, 2001). On this approach to adaptation, the project is implemented incrementally, with short development cycles (Miller, 2001) and a co-located team.

Free/open source software offers users four basic and irrevocable freedoms: to run, study, improve and distribute the program code (FSF, 2008). Access to program source code is a prerequisite for these freedoms. The FOSS development model does not comprise any technological breakthrough, but rather represents a new manner of developing software products based on special characteristics. Therefore, FOSS is also considered a development model with specific practices for the main phases of a development process (DiBona et al., 1999). It can be understood by the metaphor of the bazaar (Raymond, 2001), where the projects are developed collaboratively and transparently. Users are welcome to participate and provide feedback to the development team at any stage. Developers with different levels of skill and availability, geographically distributed around the world, work on a voluntary basis, using the Internet as a communication channel (Capiluppi et al., 2003).

From the main characteristics of the software development models presented, it can be seen that they have both similarities and significant differences. For instance, agile is more iterative and incremental than plan-driven, and focuses on customer collaboration, while plan-driven focuses more on following processes and explicitly documenting as much as possible. FOSS development is close to agile methods in terms of iterative and incremental development, but differs in its distributed (as opposed to co-located) development. In addition, FOSS needs tools to make its distributed approach work, while agile methods do not have this focus. Therefore, in this respect, FOSS tends to be more similar to plan-driven and less similar to agile methods.

Therefore, each one addresses better, or works better on, different characteristic issues in software development projects. Since any given development project may require practices from different models in order to be effective, research in the area has discussed how to accommodate the particularities of each model in development processes, so as to achieve better productivity.

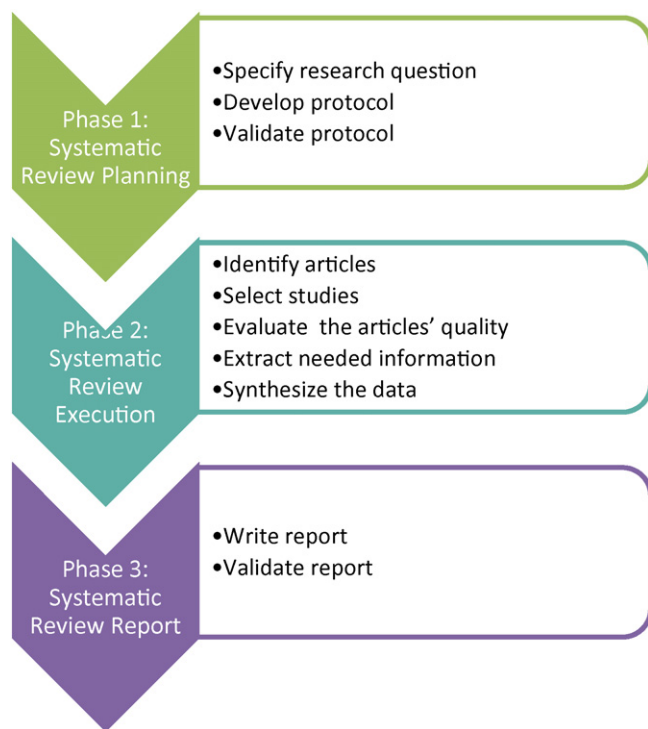


Fig. 1. Systematic review process (Brereton et al., 2007).

3. Systematic review process

A systematic literature review (Biolchini et al., 2005; Brereton et al., 2007; Kitchenham, 2004) is a type of secondary study, i.e., a study based on previously published research. It is a research methodology developed to gather, evaluate and analyze all the available literature relevant to a particular research question, topic area, or phenomenon of interest (Kitchenham et al., 2007; Kitchenham and Charters, 2007).

The main reason for performing a systematic review is to improve the quality of the material covered in the subject of interest, as compared with what is obtained by a less formal review. By preventing unnecessary duplication of effort and error it can help in planning the research, and can orient the investigation process.

In contrast with a conventional literature review conducted in an ad-hoc manner at the start of a study, a systematic review follows a strict, well-defined sequence of methodological steps (Fig. 1), which yield high scientific value and reliable results. A systematic review requires considerable effort in comparison to a conventional literature review, but offers complete, in-depth research in an area of interest (Kitchenham and Charters, 2007), ensuring important papers are not overlooked or incorrect conclusions drawn (MacDonell et al., 2010).

The concept of systematic review emerged in the field of medicine, and its adaptation to Software Engineering is presented in Kitchenham (2004). Systematic Literature Reviews have been gaining significant attention from software engineering researchers since 2004. Several researchers (Dyba and Dingsoyr, 2008; Kitchenham et al., 2009a,b, 2007; MacDonell et al., 2010; Pedreira et al., 2007) have reported their experiences and lessons learned from applying systematic reviews to different subject matter in Software Engineering (Babar and Zhang, 2009). The method proposed consists of three main phases: planning, execution and reporting. Fig. 1 gives an overview of the systematic review process used in this study. Each of the activities is described in the sections below.

During the *planning* phase, the researcher identifies the review objectives, specifies the research question, and develops the review protocol (see Section 4). At the end of this phase, the protocol must be validated before being applied, in order to establish the feasibility of the plan.

In the *execution* phase, searches are performed in the defined publication sources. The studies returned are evaluated according to the criteria established in the protocol. The relevant data found in the selected papers are extracted and synthesized.

In the third phase, the results of the review are *reported*. In practice, this phase is performed throughout the whole process, storing results from all previous phases. The whole process, including intermediate results, must be documented.

In this study, although the survey maintains the same formal characteristics of a systematic review as described above, it is considered a quasi-systematic review (Travassos et al., 2008). This is because it is an exploratory study designed to characterize the research area, and no baseline is established for comparison of the results obtained.

4. Planning phase

This quasi-systematic review begins by defining a review protocol that specifies the main and secondary research questions. The protocol should specify the inclusion and exclusion criteria and document the search strategy used in order to allow readers to ascertain the degree of accuracy and completeness.

The following subsections describe the components of the protocol used in this review. This protocol was defined based on the template proposed by Biolchini et al. (2005). The protocol was created by the authors and reviewed by an independent researcher who is a specialist in Empirical Software Engineering, including systematic reviews.

4.1. Objectives and question formulation

The objective of this review is to characterize reconciliation among plan-driven, agile, and FOSS development models. However, it was not intended to investigate the individual characteristics of each software development model or report on the use of individual approaches in software development projects. To achieve this goal, this quasi-systematic review aims to answer the following main question (MQ) and secondary questions (SQ):

MQ: *How can organizations reconcile among plan-driven, agile, and FOSS software development models?*

SQ1: *What are the opportunities and challenges in reconciliation?*

SQ2: *What proposals have been made for reconciliation?*

SQ3: *What reconciliation strategies have been adopted by organizations or communities?*

SQ4: *What results have been achieved by organizations or communities attempting reconciliation?*

4.2. Review scope

The scope of this review was defined according to the PICO (Pai et al., 2004) approach that structures the research question into four basic elements: Population, Intervention, Comparison and Outcome.

- *Population*: software development organizations and software development projects.
- *Intervention*: software development process (plan-driven, agile, and free/open).
- *Comparison*: Not applied.

• *Outcomes:*

- Characterization of reconciliation among software development models.
- Opportunities and challenges for reconciliation among software development models.
- Proposals for reconciliation among software development models.
- Strategies for reconciliation among software development models.
- Results obtained with reconciliation among software development models.

4.3. Control papers

The control papers were obtained during a previous conventional literature review. Control papers are helpful to provide an initial understanding of the area in order to define search keywords (Section 4.5). They are also used to test and calibrate the searches in the electronic databases. The three papers used as control in this search were:

1. (Boehm and Turner, 2003a) Boehm, B., Turner, R., 2003. Using risk to balance agile and plan-driven methods. *IEEE Computer*, 36(6), 57–66.
2. (Boehm and Turner, 2004) Boehm, B., Turner, R., 2004. Balancing agility and discipline: evaluating and integrating agile and plan-driven methods, *International Conference on Software Engineering (ICSE)*, 718–719.
3. (Vinekar et al., 2006) Vinekar, V., Slinkman, C.W., Nerur, S., 2006. Can agile and traditional systems development approaches coexist? An ambidextrous view. *Information Systems Management*, 23(3), 31–42.

4.4. Data sources and search strategy

The search strategy included the following electronic databases: Compendex, IEEE Xplore, Scopus, and Web of Science. These libraries were chosen because they permit article full text recovery. They also permit automated searches of article content. Although broad automated searches take more time and effort than narrow manual searches, they also return larger numbers of relevant studies (Kitchenham et al., 2009a). Therefore, these sources were considered significant in order to offer relevant content and contribute to the search result.

The ACM Library, despite its importance in the field of Computer Science, overlaps with the IEEE Xplore library and its content is also indexed by the Scopus library. Thus, although this library was not included among the selected sources, we consider that its content was covered.

4.5. Keywords

This section describes the main terms that composed each research question. The keywords were constructed considering (Kitchenham et al., 2007): terms in population, intervention, and outcomes (Section 4.2); alternative spellings and synonyms for these terms; and keywords identified in the control papers (Section 4.3).

The complete list of keywords used in this quasi-systematic review is given in Table 1, where population and intervention can be seen to be the same in the main question (MQ) and in every secondary question (SQ), since these are a subset of the main question.

4.6. Inclusion and exclusion criteria

This review includes every article returned by the protocol that meets at least one of the following criteria for inclusion (IC) and does not meet any of the criteria for exclusion (EC):

- IC1—Documents must address reconciliation between two or more software development models;
- IC2—Documents must discuss opportunities and challenges for reconciliation among software development models;
- IC3—Documents must present proposals for reconciliation among software development models;
- IC4—Documents must report experiences of organizations or communities that have implemented practices from more than one software development model.

Publications that satisfy at least one of the following EC were excluded:

- EC1—Documents not written in English;
- EC2—Documents whose full text is not available;
- EC3—Documents clearly dealing with topics irrelevant to the purpose of this review;
- EC4—Documents merely reporting the use of individual software processes in development projects;
- EC5—Documents not addressing the software development models, but tools and computing environments to support them;
- EC6—Documents addressing reconciliation, but focusing on legal aspects, business model or evaluation methods, and not on the development process itself;
- EC7—If the same study has been published more than once, the most relevant version (i.e., the one explaining the study in greatest detail) will be used and the others will be excluded;
- EC8—If a given study has been selected for a broader research question, it must be excluded from the list of selections for the narrower research question.

4.7. Selection process

Selection of articles occurred in four steps:

- (1) *Selection and preliminary organization of collected documents:* a preliminary selection of publications was made by applying the search string (Section 4.10) to selected data sources (Section 4.4). Each publication was cataloged to a reference repository for later analysis.
- (2) *Selection of relevant papers:* primary selection using the search string cannot guarantee that all material collected is useful in the research context, because application of search expressions is restricted to syntactic considerations. Thus, after the identification of publications via search engines, the documents were reviewed in view of the inclusion and exclusion criteria. One of the authors was responsible for this step.
- (3) *Evaluation of relevant papers:* the other two authors and three students evaluated the list of documents included and excluded. Where assessments conflicted, the researchers had to reach agreement on selection of publications. To reduce the risk of excluding a publication at an early stage of the study, when the researchers were in doubt the publication was included.
- (4) *Information extraction from relevant documents:* after defining the final list of relevant documents, one of the authors read them to extract information on reconciliation among software development models. This step was also evaluated by sampling under the responsibility of the other two researchers.

Table 1
Research question keywords.

PICO	MQ	SQ1	SQ2	SQ3	SQ4
Population	Organization, organisation, enterprise, corporation, business, company, industry, firm, community, project, development, engineering, software, system, application, product				
Intervention	Process, activity, method, approach, practice, methodology, technique, paradigm, procedure, principle, agile, lightweight, lean, XP, Scrum, open source, free source, libre, OSS, FOSS, FLOSS, conventional, traditional, plan-driven, plan driven, closed, rigorous, proprietary, CMM, CMMI				
Outcome	Combination, adaptation, conciliation, reconciliation, balance, aggregation, tailoring, integration, customization, coexist, juxtaposition, compatibility	Opportunity, advantage, benefit, profit, reward, strength, challenge, problem, difficulty, drawback, obstacle, trouble, weakness	Proposal, suggestion, proposition, idea	Strategy, design, plan	Result, experience, history, consequence, outcome, knowledge, conclusion, effect, solution, findings, lesson, success, failure, case study

4.8. Assessment of the quality of work

In this study, it is assumed that the document sources are reliable, and that the texts have undergone external reviews that serve as a filter assuring that they have a sufficient quality to contribute to this quasi-systematic review.

In the event of conflicting information among the documents, greater prominence will gradually be given to articles that: put forward the idea of reconciliation among models; offer a theoretical proposal for the reconciliation; and report the results of actual experiences of organizations that adopted reconciliation among software development models.

4.9. Information extraction

The data extraction strategy is designed to gather the information necessary to answer the research questions and evaluate the quality of the paper. Thus, for each document approved by the selection process, one of the authors extracts, and the others review by sampling, the information defined in the extraction form (see [Appendix A](#)). This form enabled us to record full details of the papers under review and to be specific about how each of them addressed our research questions.

4.10. Search string

According to [Pai et al. \(2004\)](#) the four basic elements – population (P), intervention (I), comparison (C), and outcome (O) – that structure the research question can be related by the AND logical operator. Since comparison (C) is not applicable to this study, this was done for the set of keywords chosen to represent population (P), intervention (I) and outcome (O), resulting in the following structure: (P) AND (I) AND (O). Within each of these three elements of the structure, their keywords were combined with the OR operator ([Table 2](#)).

Due to the large number of terms, it was decided to divide the search into multiple strings, according to possible combinations of development models. This reduced search string complexity, ensuring better readability and decreasing the probability of human and search engine error. In this case, each research question was split into four parts, as shown in the *Models* column of [Table 2](#): (a) combination of the three development models; (b) combination of agile and free/open source; (c) combination of plan-driven and free/open source; and (d) combination of plan-driven and agile.

The search strings were derived from the research questions ([Section 4.1](#)) and the keywords ([Section 4.5](#)). The strings used for the main question are listed in [Table 2](#). Since MQa, MQb, MQc, and MQd vary only in the development model in intervention, all of them have the same population and outcome. The same procedure was used in the secondary questions.

5. Execution phase

After the planning phase, the review protocol can be applied. This section describes in detail how this execution phase was conducted.

5.1. Search execution

The first round of searches in each of the selected digital libraries was performed in April 2009. In the Compendex library, the search was performed with the *autostemming* option and using the *summary* field in the search. Due to the small number of documents returned by the original string, only the population and intervention were considered to search, ignoring the result, and absorbing the effort of manually selecting documents. As the Compendex machine returned duplicate items to the same string, these duplicates were promptly excluded. Only two of the three control papers were indexed by Compendex library, and were successfully retrieved by the search.

The IEEE Xplore library returned few documents with the original string, which had to be reworded to increase its range. Thus, it was also considered to search only the population and intervention. To improve the quality of the results, the search was performed using the *summary* field. Of the two control papers indexed by the library, only one was recovered.

The Scopus database was searched using the *title*, *abstract* and *keyword* fields. The three control papers were indexed and recovered.

Lastly, the Web of Science search engine failed to process the full string. Due to implementation limitations, this machine is restricted to 50 search terms and 49 operators. Thus, the string used did not include the outcome, but was limited to population and intervention. Also, the *topic* (TS) field was chosen for the search, since it includes title, abstract and keyword fields. It was noted, using the control papers, that the number of results increased if the terms were included also in plural form. Thus, the original strings (in [Section 4.10](#)) were changed to include a simplification of the population terms, and the plural terms, while at the same time keeping to a limited number of terms.

To summarize, [Table 3](#) shows the consolidation of the original number of papers returned by searches performed for the main questions in each digital library. This total represents the raw data and is useful in order to understand how many papers were filtered in the following steps.

5.2. Elimination of duplicates

As a first step in the analysis of retrieved documents, the repetitions found in searches were eliminated, keeping the remaining paper accounted for the digital library with the largest number of items.

Table 2
Search strings.

Question	Models	PICO	String
MQa	Plan-driven, agile, and FOSS	Population	("software development" OR "software engineering" OR "software project" OR "system development" OR "system engineering" OR "system project" OR "application development" OR "application engineering" OR "application project" OR "product development" OR "product engineering" OR "product project" OR "software organization" OR "software organisation" OR "software enterprise" OR "software corporation" OR "software company" OR "software industry" OR "software firm" OR "software community")
		Intervention	AND ("open source" OR "free source" OR libre OR OSS OR FOSS OR FLOSS) AND (agile OR lightweight OR lean OR XP OR Scrum) AND (conventional OR traditional OR "plan-driven" OR "plan driven" OR closed OR rigorous OR proprietary OR CMM OR CMMI) AND (process OR activity OR method OR approach OR practice OR methodology OR technique OR paradigm OR procedure OR principle)
		Outcome	AND (combination OR adaptation OR conciliation OR reconciliation OR balance OR aggregation OR tailoring OR integration OR customization OR coexist OR juxtaposition OR compatibility)
MQb	Agile and FOSS	Intervention	AND ("open source" OR "free source" OR libre OR OSS OR FOSS OR FLOSS) AND (agile OR lightweight OR lean OR XP OR Scrum) AND (process OR activity OR method OR approach OR practice OR methodology OR technique OR paradigm OR procedure OR principle)
MQc	Plan-driven and FOSS	Intervention	AND ("open source" OR "free source" OR libre OR OSS OR FOSS OR FLOSS) AND (conventional OR traditional OR "plan-driven" OR "plan driven" OR closed OR rigorous OR proprietary OR CMM OR CMMI) AND (process OR activity OR method OR approach OR practice OR methodology OR technique OR paradigm OR procedure OR principle)
MQd	Plan-driven and agile	Intervention	AND (agile OR lightweight OR lean OR XP OR Scrum) AND (conventional OR traditional OR "plan-driven" OR "plan driven" OR closed OR rigorous OR proprietary OR CMM OR CMMI) AND (process OR activity OR method OR approach OR practice OR methodology OR technique OR paradigm OR procedure OR principle)

From the total of 1078 publications examined, 260 (24% of the total) were duplicates, and excluded. Thus, 818 papers were selected for analysis in the following stage. This percentage of repetitions is a reflection of the overlapping among documents indexed in different digital libraries. In particular, the IEEE Xplore database was observed to overlap in many cases with the Scopus and Compendex libraries. However, to ensure greater coverage, all search engines were maintained, as some papers were indexed by only one of them.

After excluding duplicates, we calculated the percentage distribution of publications found by the search engines. From Fig. 2 it can be seen that results from the Compendex library predominated.

5.3. First filter: title and abstract

After elimination of duplicates, a first assessment was made by reading all the titles and abstracts and applying the exclusion criteria. Papers with titles and abstracts that clearly indicated they were outside the scope of this quasi-systematic review were excluded. As an example, since our search strategy included the term "xp and software", we got several hits on Microsoft's Windows XP operating system.

In addition, titles and abstracts are not always clear indicators of what a paper is about. If it was unclear whether a study matched the screening criteria, it was included for detailed quality assessment in the next stage.

Table 3
Overall results of the main question searches.

Library	MQa	MQb	MQc	MQd
Compendex	17	55	171	193
IEEE Xplore	4	10	36	100
Scopus	14	56	19	73
Web of Science	4	30	184	112
Total	39	151	410	478

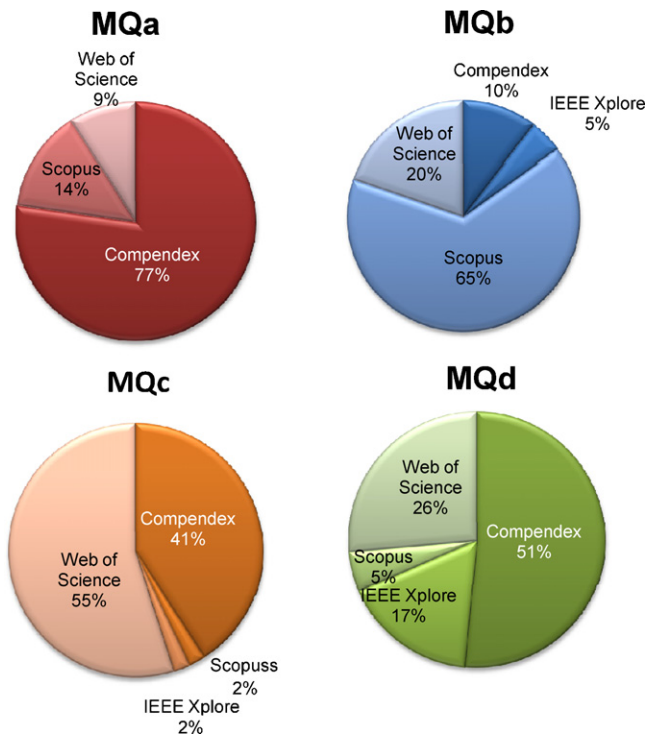


Fig. 2. Distribution of search engine results.

From the total of 818 papers analyzed at this stage, 610 were excluded and 208 items were selected for analysis in the next phase, representing 25% of the studies analyzed in this phase (Table 4).

After application of the first filter by one author, the list of excluded items was revised by five other researchers (the other two authors and three students). The purpose of this validation is to ensure that items are not excluded improperly, and that relevant work is not lost. So, in the event of doubt or deadlock between

Table 4
Results of searches after the first filter.

Library	MQa	MQb	MQc	MQd
Compendex	6	2	42	63
IEEE Xplore	0	1	4	22
Scopus	2	16	3	4
Web of science	0	1	37	5
Total	8	20	86	94

Table 5
Results of searches of the main question after the second filter.

Library	MQa	MQb	MQc	MQd
Compendex	1	0	3	17
IEEE Xplore	0	0	1	6
Scopus	0	3	0	1
Web of science	0	1	6	3
Total	1	4	10	27

Table 6
Results of secondary question searches after the second filter.

Models	SQ1	SQ2	SQ3	SQ4
(a) Plan-driven, agile, and FOSS	1	1	0	0
(b) FOSS and agile	2	2	1	1
(c) FOSS and plan-driven	2	6	0	0
(d) Plan-driven and agile	9	18	10	10
Total	14	27	11	11

researchers, the publication was included in the list of selected papers. The results shown below were consolidated after this validation.

5.4. Second filter: paper recovery and reading

Accurate, detailed assessment requires reading the full text of papers. So we started recovering the files with the complete paper texts. However, it was not possible to access all publications, even after contacting the authors by e-mail. Of the 208 items that should have been assessed at this stage, full texts of 182 (88%) were recovered through search engines or personal e-mail contact with the authors. Accordingly, it was not possible to access the full text of 26 papers.

The papers retrieved were read in order to select candidates for systematic review (Table 5). All papers were analyzed according to the inclusion and exclusion criteria. The complete list of the 42 papers selected is detailed in Appendix B. As regards the secondary questions, the results of applying the second filter are summarized in Table 6, where the same paper may address more than one SQ.

5.5. Information extraction

During this stage, all the information necessary to answer the research questions was extracted from the 42 studies selected in this review. The information was captured from each selected paper, as in the form shown in Appendix A. Extracted data were held in tables, one file per paper. All the resulting forms were grouped by the research questions they address.

5.6. Instruments

All references retrieved were imported into JabRef,¹ the reference manager used as the main tool in this quasi-systematic review.

¹ Site JabRef: <http://jabref.sourceforge.net/>.

The tool was useful to identify duplicates. In addition, it permitted custom fields to be created to register the extracted information, to maintain the track of the source database and to control the decision at each phase of the filtering process.

JabRef was also used to perform secondary question searches within the main question results. It also offers the possibility to create specific reports to export all the information in a predefined format to facilitate the work of handling references and information analysis.

6. Reporting phase

This section reports and discusses the results obtained by the quasi-systematic review. Initially, execution of the protocol in the search engines returned a total of 1078 publications. These papers were submitted to a filtering process, comprising four steps, as described in the sections above: elimination of duplicates, filtering by title and abstract, filtering by availability of full-text, and filtering by reading the full text of the paper. Fig. 3 shows the number of items returned from each digital library during each step of the filtering process.

Finally, 42 papers were actually selected as the result of the quasi-systematic review execution. This means that only 3% of the documents initially obtained actually contributed to the quasi-systematic review. The vast majority of papers read were excluded for lack of any direct contribution to characterizing reconciliation among software development models.

Of the exclusion criteria that were applied, the main reasons for excluding studies were: focus on individual software development processes only (EC4), lack of relevance to the study theme (EC3), and focus on tools and computing environments rather than software development models (EC5).

The citation among the 42 papers obtained during the quasi-systematic review, is shown in Fig. 4. This graphical representation helps to identify the most influential publications. The most-cited papers are highlighted, showing clearly the influence of Boehm and Turner's papers on this research topic. Although no time restriction was placed on the search, the results date from only 2002, showing the lack of maturity of this research field and the need, underscored by some authors, for more scientific papers on the topic. The field is also recent because of the initial perception that the plan-driven, agile, and FOSS models were opponents, and that there was no room for any attempt to integrate them.

6.1. Threats to validity

For a complete analysis of the results obtained with this quasi-systematic review, it is also important to consider the limitations of this review.

In this regard, the main threat to validity identified is that other relevant works are not indexed by the main search engines, due to the lesser standing of the publication. For instance, some papers (Glass, 2001; Glazer et al., 2008; Warsta and Abrahamsson, 2003) were identified in the conventional literature review and used as basis for preparing the research protocol, although they could not be used as control papers to calibrate the searches in search engines.

Regarding internal threats to validity, it is important to consider that data extraction was performed by only one researcher, which may entail some risk of bias. Although three students were involved in the paper evaluation filter, they did not participate in the data extraction step, which the other authors reviewed by sampling.

Finally, as mentioned in Section 5.4, another threat to validity is that it was not possible to access the full text of 26 papers, despite efforts to contact authors by e-mail. These represent approximately 2% of all publications retrieved, and some of them, after the last

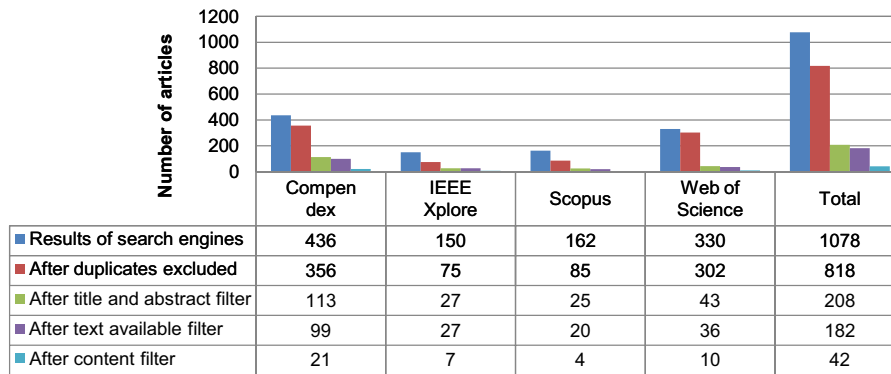


Fig. 3. Filtering process.

analysis, might have contributed to the results of this systematic review.

7. Reconciling software development models

The purpose of this section is to characterize reconciliation among software development models, according to the results of the quasi-systematic review. Since the research question was further divided into a set of sub-questions, each of these questions is

discussed and analyzed individually. Finally, some general conclusions are drawn.

7.1. MQ: how can organizations reconcile plan-driven, agile, and FOSS software development models?

This quasi-systematic review yielded a number of important observations. First of all, *in the technical literature, there is a scarcity of reports about reconciliation among all three software development*

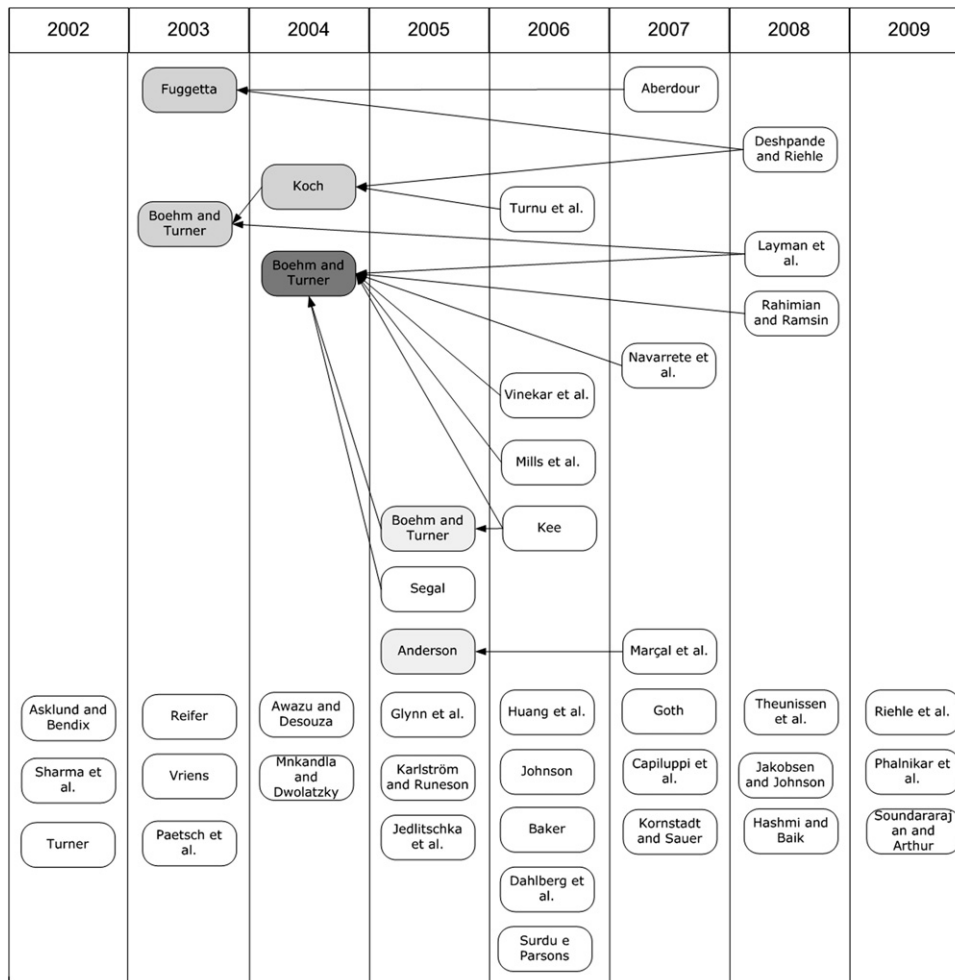


Fig. 4. References among the papers selected.

models. By the end of the review, only a single paper (Theunissen et al., 2008) had been found covering all of them. Theunissen et al. (2008) discuss the challenges for reconciliation between agile and FOSS development models within a corporate culture. The type of reconciliation considered in the paper concerns an agile team working within an organization and collaborating with an outside FOSS project. In fact, the paper does not exactly focus on the three software development models, since the plan-driven view is not embodied in the process, but in the environment and organizational culture. In addition, this reconciliation does not happen with the three models being used within the same organization.

This raises the question: why are there so few studies correlating the three software development models? After some reflection, the first possibility to be considered is that not all authors and researchers see a development process in FOSS projects, considering that these projects are conducted in an ad-hoc manner. However, as discussed in Section 2, we do not support this idea and have decided to look for other alternatives.

The most reasonable alternative answer to this question is that reconciling only two software development models is already a major challenge. This research issue is quite recent, given that preliminary research on reconciling models was published in 2002, describing proposals to combine only two models, before engaging a third complicating factor in this equation.

This hypothesis is also underpinned by a second observation: there is more work to combine the plan-driven and agile models. Since, for some time, the plan-driven model was the established standard in many organizations, it is the starting point for many reconciliation initiatives. Therefore, that effort to combine plan-driven and agile models should predominate is justified by the fact that both are in widespread use in organizations and there is a common sense in the field that agile and plan-driven approaches are poles apart, which makes them a focus and motivates research. Therefore, this could be a progressive way to reach reconciliation of others.

Another observation is that *reconciliation between the agile and FOSS development models is still incipient*. In general, it is based on integrating one particular practice, like test-driven development into a FOSS project. Although this combination is not yet widely explored in literature, some authors (Simmons and Dillon, 2003; Warsta and Abrahamsson, 2003) emphasize the proximity and similarities of these two models and consider that the FOSS model may lie between the plan-driven and agile models (Abrahamsson et al., 2002). For example, both agile and FOSS approaches emphasize iterative development and frequent releases, with minimum documentation and a focus on testing (Barnett, 2004). However, they differ in one significant respect: several agile methods emphasize co-location and face-to-face development, while FOSS typically involves geographically distributed developers. In addition, FOSS needs tools to make its distributed approach work, while agile methods do not. Therefore, FOSS is similar to plan-driven in all the respects it is dissimilar to agile. Other similarities and differences between the two models are described by Tsirakidis et al. (2009).

Finally, the results of this quasi-systematic review also indicate that *organizations can reconcile the three development models at three different levels: organizational, group, or process*. At the *organizational* level, the organization can have independent development teams, segregated in different organizational units, each of which uses one of the three development models. This allows the organization to work with complete as well as hybrid methods, and does not pose the organizational culture challenge. At the *group* level, each development team can switch between different models depending on the project context. This assumes that any organizational culture issues have been overcome and both complete and hybrid models can be used. At the *process* level, a new hybrid

process can be created by combining various parts of each method. This only allows hybrid methods, and assumes that cultural challenges can be overcome to some extent. These levels will be used to categorize the proposals (SQ2), strategies (SQ3) and experiences (SQ4).

7.2. SQ1: what are the opportunities and challenges to reconciliation among plan-driven, agile, and FOSS software development models?

To answer this secondary question, we analyzed opportunities and challenges separately. Accordingly, the 14 papers identified (Table 6) for SQ1 are distributed in Tables 7 and 8. Note that these opportunities and challenges refer to reconciliation among models, since this study is not concerned to investigate the potential benefits or problems of individual models.

The main opportunities for reconciling plan-driven, agile, and FOSS development models are summarized in Table 7 and ordered according to the number of citations. These results indicate an *interest in collaboration*. Reconciliation can stimulate people's involvement in the development process, allowing them to meet and collaborate to facilitate group efforts in the future. Also, as regards reconciliation of FOSS with plan-driven or agile development, there is also the possibility of collaborating with open source communities with large pools of developers available.

In addition, availability of code is seen as an opportunity for innovation in software development. Other opportunities mentioned, such as quality improvement and cost reduction, are common arguments for research into software engineering processes and software development in general.

The main challenges to reconciliation among the three software development models are summarized in Table 8, which shows a diversity of problems that organizations and researchers must address to make reconciliation feasible. Not surprisingly, the most cited challenge is *the barrier to changing organizational culture*. When teams are used to working with one development model or when the organization has a culture that favors practices from one specific model, reconciliation can be a major challenge.

The need to review the number of artifacts usually created during a plan-driven project is also pointed to as an important challenge to reconciliation, since agile and FOSS development emphasize simplicity and parsimony in documentation. Furthermore, in plan-driven development, knowledge is considered a scarce resource that must be protected, and a source of power. Thus, it is hardly shared, but restricted to certain participants. FOSS projects, on the other hand, are based on free exchange of knowledge through information and communication technology.

7.3. SQ2: what proposals have been made for reconciliation among plan-driven, agile, and FOSS software development models?

The main proposals for reconciling software development models have been categorized and are presented in Table 9. Although each proposal is briefly summarized, it is not the purpose of this section to discuss their validity or importance, since this is an exploratory study to achieve a better comprehension of the research field.

The most common type of proposal is one that suggests adapting reference models – such as the staged model for software evolution (Capiluppi et al., 2007), PROFES (Product Focused Improvement of Embedded Software Processes) (Jedlitschka et al., 2005) or Six Sigma (Hashmi and Jongmoon Baik, 2008) – to introduce agile or FOSS development practices into specific contexts like COTS reuse (Navarrete et al., 2007), distributed development (Kornstädt and Sauer, 2007), or mobile applications (Rahimian and Ramsin, 2008).

Table 7
Key opportunities for reconciliation among software development models.

#	Opportunities	Description	Papers
1.	Collaboration	Facilitates collaboration among participants in the process or offers the possibility of collaborating with open source communities with large pools of developers available	[5,12,30]
2.	Availability of code	Availability and transparent access to source code favors innovation	[12,14]
3.	Cost	Reducing project costs	[12,25]
4.	Quality	Improving code quality	[4]

In proposals of this kind, adaptation focuses on the whole model and aims to produce a new hybrid model designed for a specific context.

Application of specific practices, such as configuration management (Askund and Bendix, 2002), continuous code integration (Deshpande and Riehle, 2008), knowledge management (Awazu and Desouza, 2004), test-driven development (Turnu et al., 2006), and peer review (Johnson, 2006), from one model to another is also a widespread kind of approach, probably because it reduces the reconciliation problem to a particular practice. In this case, it is easier to plan, implement, monitor and control the results of such reconciliation. This kind of proposal does not aim to define a new hybrid model. It investigates the effect of using these individual practices in a project guided by a different model.

It can be seen from Table 9 that this application of practices usually involves two development models. Askund and Bendix (2002) compare configuration management (CM) in plan-driven and FOSS contexts in order to understand the best practices applied in each and to discuss how these can be transferred between them to mutual benefit. They do not discuss CM in an agile context. This makes sense because agile methods do not address any specific practice to CM and do not emphasize the use of tools, which are very important to CM work. On the other hand, agile is concerned with adapting to changes and product integrity and, therefore, some CM practices are also needed. Accordingly, agile would also benefit

from the analysis and transfer of some plan-driven and FOSS CM practices, although these practices would need some adaptation, as discussed in Moreira (2009).

Continuous code integration, a common practice in the agile world, was also investigated in the context of FOSS projects (Deshpande and Riehle, 2008). Their results are inconclusive, as it is not possible to say whether FOSS projects do not adopt continuous code integration on a significant scale or FOSS projects have always practiced continuous integration. In this case, the advent of agile methods did not bring any changes, simply because they had arrived in open source software development long before. There is thus a need for research on this topic to be broadened and extended to the context of plan-driven projects.

FOSS communities were also examined for insights into how to expand current knowledge management practices in organizations (Awazu and Desouza, 2004). In traditional knowledge management, knowledge is considered a valuable, rare, inimitable, and non-substitutable organizational resource that contributes to sustained competitive advantage. In this mindset, knowledge is a scarce resource, so it must be protected. On this view, organizations share knowledge only on a need-to-know basis. The rigid structure of knowledge management discourages innovation and offers poor incentives to reuse knowledge. However, these problems traditionally encountered by knowledge management efforts in organizations are, for the most part, absent in FOSS communities. FOSS communities have large populations of developers, who

Table 8
Key challenges for reconciliation among software development models.

#	Challenges	Description	Papers
1.	Organizational culture	Cultural barriers – customers, managers, and developers may not accept development model change	[1,20,24,29,42]
2.	Number of artifacts	The reputation for copious documentation (plan-driven) contrasts with the simplicity and parsimony of creating only the really necessary artifacts (FOSS and agile)	[29,37,39,42]
3.	Knowledge management	Explicit knowledge (plan-driven) <i>versus</i> tacit knowledge (FOSS and agile)	[14,39]
4.	Organizational structure	Hierarchical control (plan-driven) <i>versus</i> flexibility on a voluntary basis (FOSS) or self managed teams (agile)	[24,39]
5.	Recognition and reward system	The organization recognition and reward system is usually based on individual assessment (plan-driven and some FOSS projects) and must be reviewed to stimulate collaboration and teamwork	[24,39]
6.	Communication	Face-to-face communication (agile) <i>versus</i> communication over the Internet and distributed development (FOSS and plan-driven)	[1,20]
7.	Decision making	Centralized (plan-driven) <i>versus</i> decentralized decision making (FOSS and agile)	[1,24]
8.	Quality assurance	Peer review (FOSS and agile) <i>versus</i> formal techniques that make non-conformances visible to senior management (plan-driven)	[1,25]
9.	Requirements	Complete and documented requirements (plan-driven) <i>versus</i> functional and informal (FOSS and agile) requirements	[31,37]
10.	Planning work	Documented and formal planning (plan-driven) <i>versus</i> informal planning (FOSS and agile)	[1]
11.	Monitoring work	Monitoring activities carried out by developers (plan-driven) <i>versus</i> voluntary work (FOSS) or teams controlling their own tasks (agile)	[1]
12.	Customer relationship	Need to take actions to prepare the client for the new development process	[39]
13.	Technology	Appropriateness of existing technologies, environments and tools to deployment of the new development model	[39]
14.	Keeping certified processes	Organizations are concerned to improve their software development processes and, at the same time, to maintain certification of their processes (plan-driven)	[31]

Table 9
Main proposals for reconciliation among software development models.

#	Level	Proposal	Description	Papers
1.	Process	Adaptation of reference models	This type of proposal involves adapting traditional reference models to incorporate characteristics of agile development so that they can be applied to specific contexts, such as COTS reuse, distributed development, or mobile applications.	[6,10,17,27,33,35,36]
2.	Process	Application of specific practices	Comparison and transfer of specific practices from one model to another: Configuration management – plan-driven and FOSS. Continuous code integration – FOSS and agile. Knowledge management – plan-driven and FOSS. Peer review – plan-driven and FOSS. Test-driven development – FOSS and agile.	[3,4,8,11,14]
3.	Process	Combination of practices from different development models	This type of proposal involves mapping equivalence between practices of different development models, such as XP or Scrum (from agile) with CMMI or ISO 12207 (from plan-driven). In general, this mapping results in an analysis of how agile methods adhere to plan-driven maturity models and how agile methods can be modified to incorporate the practices that are missing.	[19,25,32,34]
4.	Group	Risk analysis	This proposal from Boehm and Turner includes using risk analysis to balance agility and discipline in software development projects, taking into account the characteristics and needs of the project	[22,40]
5.	Process	Statistical control	This proposal involves using objective mechanisms for measuring and understanding natural process variations, and statistical analysis to determine whether the process is under control, that is, if the process is running according to plan	[38]
6.	Organizational	Organizational structure	This proposal provides an organizational structure to accommodate organizational cultures for plan-driven and agile software development by separating development models into different organizational units subordinated to a common management	[24]
7.	Process	Process improvement framework	This proposal argues that CMM can be used as a framework for improving processes to accommodate XP practices. The infrastructure and culture of CMM processes can help institutionalize XP in organizations. The CMM management practices may also help overcome the scalability problems of XP	[42]

share common interests and backgrounds, and are available to give rapid feedback and immediate solutions to specific problems. They also have experts motivated by the idea of forming part of core group and gaining ultimate peer recognition. Although not mentioned in the paper, in agile development a great deal of product knowledge is tacit and shared among project members through face-to-face communication and personal interactions. It would thus also be interesting for future studies to investigate how these agile practices can contribute to improving knowledge management in organizations or in FOSS communities. In addition, the use of explicit knowledge management practices, coming from plan-driven and FOSS development, should be carefully analyzed to ascertain whether it shows any potential to be beneficial in an agile context.

As regards test driven development, the goal of the work is to study the effects of adopting agile practices in FOSS development (Turnu et al., 2006). In particular, Turnu et al. started evaluating test driven development because they considered it easier to apply in a distributed environment than most other agile practices. Using a simulation modeling approach, they concluded that this practice yields better results in terms of code quality. However, real case studies are necessary to confirm these results and to extrapolate the use of this practice to a plan-driven context.

Another type of proposal presented in literature combines practices from different models, such as XP or Scrum with CMMI or ISO 12207 (Marcal et al., 2007; Vriens, 2003). This generally involves mapping equivalence between practices from these models. It results in an analysis of how agile methods adhere to plan-driven maturity models and how agile methods can be modified to incorporate the practices that are missing. This type of reconciliation proposal usually does not consider FOSS development, because it has no formal model or complete definition that can be used to compare and fit agile methods or plan-driven reference models. Although this kind of proposal also results in definition of a hybrid

model, it differs from the adaptation of reference models, because the model here is generic to software development and not specific to a context like COTS or mobile.

However, the complex nature of software development and the wide variety of existing models (XP, Scrum, Crystal, FDD, Lean, CMMI, ISO 12207, ISO 9001 – to mention just some) make this comparison an arduous and imprecise task (Boehm and Turner, 2003b). Furthermore, this kind of integration among development models limits the potential for future synergy among them, because it builds an incomplete hybrid model, and it cannot be guaranteed that the resulting process actually embodies the characteristics or philosophies of the original models.

The proposal of Anderson (2005) also refers to the process level, but has a different focus. This proposal is based on the idea that maturity models are not prescriptive and their practices are only indicative. Therefore, it is possible to use alternative practices that facilitate compatibility with agile methods. The key point is to provide appropriate evidence of a practice's effective implementation in an organization. This interpretation is not always applied in certified organizations. It can be done using Deming's notion that quality refers to conformance to process. This conformance can be checked through an understanding of natural variation in processes and by statistical analysis to verify that processes are under control (see more details in Florac and Carleton, 1999), considering metrics that make sense in both the plan-driven and agile worlds. The use of FOSS processes is not discussed, but these principles can be applied to any development model.

Boehm and Turner's (2003a, 2004) proposal suggests risk analysis of project characteristics as a way to select the appropriate method and achieve balance between agility and discipline. This analysis considers three categories of risk: environment risks, the risks of adopting an agile model, and the risks of adopting a plan-driven model. On the citations map (Fig. 4) this proposal proved to be the most referenced by other researchers.

Table 10

Strategies for reconciling software development models in organizations or communities.

#	Organizations or communities	Levels	Strategies	Positive results	Risks	Papers
1.	ABB automation and ericsson microwave	Process	Adaptation of the <i>stage-gate</i> model to incorporate agile practices from XP.	+ Early feedback + Important features not delayed + Micro-planning avoids rigid plans + Internal communication improved + Product quality + Concrete measures of development project progress + Developer motivation increased	– Little long-term planning – Agile team isolation within the organization – Managers feel threatened – Training needs	[26]
2.	BMW Car IT	Process	Adaptation of a traditional process improvement model (PROFES) for agile development, combining XP, Scrum and FDD	+ Exploration of customer quality requirements + Development of a learning organization + Available resources invested in product quality-related processes in a goal-oriented manner in line with organizational needs + Customer integration	– Customer unable to articulate quality requirements explicitly	[17]
3.	DTE Energy	Process	Use in-house agile initiative for process improvement designed to combine CMMI Level 3 with agile methods	Paper reports IT organization's journey towards combining CMMI and agile, but does not present later results. Summarizes some recommendations for other organizations interested in taking this approach		[29]
4.	Memphis University	Group	Experience report on proposed risk analysis that resulted in choice of XP for development project in question.	+ Decision based on project characteristics	– Lack of team training and experience	[28]
5.	Microsoft	Process	Use of statistical control for integrating CMMI level 5 with agile development.	+ Develops a truly agile full life-cycle process + Meets requirements for all 5 CMMI model levels + Adheres to Agile Manifesto + Process heaviness was reduced by around 85% + Defines metrics useful to agile development	– Hard work to process design	[38]

6.	Motorola Wireless	Process	Combination of CMMI level 5 with agile methods reduced defects and improved productivity.	+ Improved quality - defects reduced 75% + Improved cycle time + Reduced risk + Productivity improved threefold + Staff effort reduced 59.6%	– Agile scaling up – Potential for ambiguous requirement statements if customer and team fail to collaborate – Customer cannot be on-site full time in large distributed projects – Team members and customers may not accept process paradigm shift – Lack of senior management involvement – Lack of agile training and mentoring – Difficult to implement and to maintain focus on using the practice	[30]
7.	Philips	Process	Combination of CMM level 2 and ISO 9001 with XP and Scrum	+ ISO 9001 Certification		[25]
8.	PyPy community	Process	FOSS project using an agile practice: sprint-oriented development.	+ Technical progress		[5]
9.	Systematic	Process	Combination of Scrum and CMMI models.	Make recommendations to activities an agile project could consider adopting. Activities inspired by mandatory goals and expected practices from subset of CMMI process areas		[34]
10.	U.S. Army	Process	Combination of CMMI level 5 with XP, selecting best practices for institutionalization.	+ Not blind adoption of all agile approach techniques + Best method depends on nature of project	– Difficult to educate user representatives on agile methods	[21]
11.	Volvo Technology	Process	Combination of ISO 15504 and XP models.	Findings discussed by key issue: management and organizational, people, processes and technology (see details in paper)		[39]

The organizational structure proposal (Vinekar et al., 2006) is the only one that refers to organizational level. It attempts to accommodate the organizational culture of plan-driven and agile software development by separating development models into different organizational units subordinated to a common management. It separates the development teams of different development models, and each project is developed in the unit with characteristics that fit the project most closely. The idea of organizational separation can also be applied to create a unit dedicated to FOSS development.

Finally, the process improvement framework proposal (Reifer, 2003) argues that CMM can be used to introduce XP practices in organizations. The infrastructure and culture of CMM processes can help institutionalize XP. In addition, CMM management practices may also contribute to overcoming XP scalability limitations. Even though Reifer claims that these points could help reconciliation, he acknowledges that it is not what is happening in practice and reports some resistance to change (which is a major challenge discussed for several authors, as presented in SQ1).

7.4. SQ3: what strategies are adopted by organizations or communities for reconciliation among plan-driven, agile, and FOSS software development models?

7.4.1. SQ4: what results have been achieved by organizations or communities that have attempted reconciliation among plan-driven, agile, and FOSS software development models?

The answers to SQ3 and SQ4 were consolidated and are presented in Table 10, which shows a significant number of large companies interested in this topic. The Levels column indicates the levels at which reconciliation (organizational, group or process) was pursued. As can be seen, almost all the strategies adopted focus on the process level. The predominant strategy adopted is a combination of practices from different models, like CMMI or ISO with XP and Scrum.

Interestingly, in the scope of this review, experiences with reconciliation at the organizational level were not identified. This can indicate that organizations are not investing on separating teams by development model. There is only one experience report, of risk analysis using Boehm and Turner's (2003a, 2004) proposal, that pursued reconciliation at the group level. After an analysis of project characteristics, a pure agile approach was used. However, this decision was not based on segregated organizational units, as the same team was prepared to work with both plan-driven and agile methods. Agile choice was determined considering project needs, which appears to be a promising path to reconciliation in the future.

Notice also that only one experience reports reconciliation of FOSS. PyPy is a FOSS project that adopted an agile practice – sprint-oriented development – to address the problem of distributed development.

The main achievements from the organizations' strategies were separated (above) into positive results or risks. Although the reports tend to emphasize the positive results, they seem to suggest that model reconciliation is a viable approach and a good strategy for achieving better quality and productivity results. Some of the risks identified echo the main challenges cited in SQ1. Finally, some of the studies (as in the case of DTE Energy) do not analyze the results objectively, but provide recommendations for how to conduct the reconciliation in practice.

8. Conclusions

This paper presented a quasi-systematic review to characterize reconciliation among software development models. Systematic reviews are useful to identify and consolidate work to date and

guide future research. The conclusions reached are an important step towards expanding the body of knowledge about reconciliation among software development models. Based on these results, some directions for future research are outlined in Section 8.2.

The main conclusion of this study is that there are three different levels of reconciliation: organizational, group, and process. Each level requires different approaches for merging methods and/or facing cultural issues.

The results also show that, to date, few studies have combined plan-driven, agile, and FOSS development models. Moreover, the larger number of works combines plan-driven and agile models. Reconciliation between agile and FOSS models is still incipient and generally based on integrating one particular practice in the context of a FOSS development project.

The main opportunities for reconciliation derive from collaboration and code availability. Collaboration can stimulate people's involvement in the development process and offers the possibility of interacting with open source communities with large pools of available developers. The code availability favors innovation in software development.

Proposals are most commonly of three types: (i) adapting traditional reference models by introducing agile or FOSS practices in specific contexts; (ii) proposals for the transfer of specific practices from their original model to another are also widespread, probably because they simplify the reconciliation problem to a particular practice; and (iii) the combination of practices from different models, comparing and mapping equivalences between practices from these models.

Finally, we detected keen interest in reconciliation on the part of large organizations. However, they are attempting to combine typically plan-driven and agile models, without the help of detailed guidelines. Their reports indicate that reconciliation among models is a viable approach and a rewarding strategy for achieving better quality and productivity results.

Despite existing proposals, reconciliation among software development models to achieve a balanced mix of plan-driven, agile, and FOSS is still an open issue, because it is an emerging area and research on most of the proposals is at an early stage. There is a diversity of challenges that organizations and researchers must address to make reconciliation feasible. The most salient is to overcome barriers to culture change. The number of artifacts usually created during a plan-driven project is also a point of divergence, as agile and FOSS development emphasize simplicity and parsimony in documentation. We argue that knowledge of these challenges can help the development of strategies to overcome them in future.

We believe that this characterization is important for practitioners who want to stay current with the state of research. This review will also assist the scientific community working with software development processes to build a common understanding of the challenges that must be addressed, as well as to identify topic areas that have been researched or where research is still lacking. Finally, the results of this kind of investigation are relevant to software industry, which is calling for solutions in this area.

8.1. Limitations

This section discusses the main limitations of this research, including the threats to validity already mentioned in Section 6.1.

The first limitation refers to the automated search used. Although this type of broad, automated search tends to find more prominent studies (Kitchenham et al., 2009a), it is also restricted by the standing of the publication, i.e., even relevant papers cannot be captured if they appeared in publications that are not indexed. This limitation can be overcome by the use of manual searches in major conferences, at least to complement the results of the automated

search. The unavailability of a few papers also prevents them from contributing to the topic, even if they have potential to do so.

Data sources such as Google Scholar, and academic theses and dissertations on the topic, are – perhaps conspicuously – not used by this study. As explained in Section 4.4, we selected data sources that allowed us to make automated searches of their content, and this was not the case with Google Scholar or the body of knowledge contained in theses and dissertations. The protocol could not be executed in Google Scholar, which even in advanced search mode does not admit the whole search string. Although to date there is no database that indexes theses and dissertations and can be used for automated searches, such theses and dissertations often do generate papers that can be captured in the databases that were used.

Finally, as mentioned in Section 7, as an emerging field, there is a scarcity of works addressing reconciliation among plan-driven, agile and FOSS development models. Even so, this paper offers many important observations that represent a significant starting point for future research on this topic, as presented in the following section.

8.2. Directions for future research

The first opportunity for future research lies in re-execution of the protocol, to capture references to more recent work that extends the search space chronologically. This could also include other search engines, such as ACM (Association for Computing Machinery), in an attempt to retrieve documents only indexed by these machines, which would extend the search space geographically. Finally, the search can also be expanded with manual searches to include: books; conferences; theses and dissertations; technical reports; and other search engines, such as Google Scholar and AISel (Association for Information Systems Electronic Library). Although the systematic approach adopted ensures the reliability and completeness of this study, it can be amplified by these extensions. In Section 7 (SQ2), we discussed how studies that showed how to combine two development models could be extended to the third. This discussion included ideas like: (i) the reconciliation of FOSS and plan-driven configuration management practices to agile model; (ii) the need for further studies on the reconciliation of the practice of continuous code integration between agile and FOSS models and extension of this search to the context of the plan-driven model; (iii) investigate how the knowledge management practices in agile model can contribute to improve knowledge management in organizations or in FOSS communities; (iv) analyze if the use of explicit knowledge management practices, coming from plan-driven and FOSS development, can be beneficial in an agile context; (v) extrapolate the use of test driven development to a plan-driven context. These ideas are all opportunities for future research. An area that still deserves to be explored is the search for studies to reconcile the three models of software development, since only one study was identified in this quasi-systematic review. First of all, it can be present in areas that were not the focus of this work. Some of these understudied areas are indicated below to guide future research. In addition, as stated before, the reconciliation research area is still at an early stage, but it is expected that in future there are more studies and results on the topic to be investigated. Finally, there also remains the possibility that organizations or other researchers have already achieved positive results on the reconciliation among agile, FOSS, and plan-driven models, but have not written about it yet.

Another interesting topic that deserves attention in future research is the emerging body of literature on agile and distributed development (Angioni et al., 2005; Ramesh et al., 2006). The coming together of these two worlds was not explored in this study, but we consider that they can also contribute to

facilitate reconciliation among plan-driven, agile and FOSS development models. Likewise, the idea of processes diversity (where different processes may be running concurrently on the same project—in multi-team projects or changing over time during the phases of the development and maintenance cycle) (Deck, 2001; Lindvall and Rus, 2000; Siebel et al., 2003) also should be investigated in future research, since the need to manage this diversity can be an important motivation for the reconciliation among software development models. Finally, approaches that deal with reconciliation considering organization and project contexts and needs, appears to be a promising path for reconciliation in the future (Jaufman and Munch, 2005; Xu and Ramesh, 2008). This is the focus of the next section.

8.3. Future work

Our research agenda on this topic involves investigating software development process tailoring (Pedreira et al., 2007) as a way to achieve a balance between characteristics of software development models. Process tailoring is the act of particularizing a general process description to derive a new process applicable to a specific situation (Ginsberg and Quinn, 1995).

We claim that it is necessary to tailor these processes to suit specific project and organization contexts. This tailoring should also consider the key distinctive features of plan-driven, agile and FOSS models: collaboration and discipline (Magdaleno, 2010a).

Collaboration can be defined as a group of two or more people working to achieve a common goal (Vreede and Briggs, 2005). Collaboration is an important factor for software organizations to achieve their productivity, quality and knowledge-sharing goals. In particular, software development is a complex process that involves collaboration among several people over time to achieve a common goal (Cugola and Ghezzi, 1998). Therefore, software development is a typical example of collaborative work (DeMarco and Lister, 1999). Discipline, meanwhile, relates to the planning level adopted in software process definition and the rigidity of control employed in process execution (Boehm and Turner, 2003b).

Both are complementary and essential in any project, but in differing proportions, depending on project characteristics. For a balanced mix between collaboration and discipline, it is necessary to understand how these aspects vary and distinguish the software development models (Magdaleno et al., 2010). In order to facilitate process tailoring, it is possible to support the project manager by automating some of the steps, possibly reducing the effort required to conduct this activity and improving the quality and appropriateness of the resulting process (Park et al., 2006). Thus, we intend to develop an infrastructure using an optimization-based approach (Magdaleno, 2010b). Finally, it is also important to investigate whether the results of this quasi-systematic review are consistent with what is observed in practice, i.e., in organizational routine. To accomplish this, we intend to plan and conduct an experimental study. This experimental study will involve a survey of industry and academic experts to validate the results and discuss the conclusions. A similar strategy was applied by (Bekkers et al., 2008) to determine the factors that most influence the selection of software development model.

Acknowledgments

This research was partially funded by CNPq under process No. 142006/2008-4 and No. 310776/2009-0. The authors would like to thank: Prof. Guilherme Travassos for his careful revision of the protocol; Vanessa Nunes (PhD student), Taisa Santos, and Alice Leite (MSc students), who helped validate this quasi-systematic review; and anonymous reviewers whose comments significantly contributed to the quality of this work.

Appendix A. Extraction form

Table 11 presents the information extraction template.

Table 11
Extraction form.

1.	Reference information: Title of document: Author(s): Publication date: Source:
2.	Information from the main question: Software development processes: Practices: Comparison criteria:
3.	Information from SQ1: Opportunities: Challenges:
4.	Information from SQ2: Proposal description:
5.	Information from SQ3: Strategies:
6.	Information from SQ4: Results:

Appendix B. Studies included in the review

Table 12 lists the 42 papers selected for this quasi-systematic review. The following information was gathered for each publication: title, database from which this publication was selected, and the inclusion criteria applied.

Table 12
Papers selected in the review.

#	Question	Title and reference	Library	Inclusion criteria
1.	MQa	Corporate-, agile- and open source software development: a witch's brew or an elixir of life? (Theunissen et al., 2008)	Compendex	IC1, IC 2, IC3
2.	MQb	Agile principles and open source software development: a theoretical and empirical discussion (Koch, 2004)	WebOfScience	IC1
3.	MQb	Continuous integration in open source software development (Deshpande and Riehle, 2008)	Scopus	IC1, IC3
4.	MQb	Modeling and simulation of open source development using an agile practice (Turnu et al., 2006)	Scopus	IC1, IC2, IC3
5.	MQb	Sprinting towards open source development (Goth, 2007)	Scopus	IC1, IC2, IC4
6.	MQc	A development process for building OSS-based applications (Huang et al., 2006)	WebOfScience	IC1, IC3
7.	MQc	A framework for creating, hybrid-open source software communities (Sharma et al., 2002)	WebOfScience	IC1, IC3
8.	MQc	A study of configuration management in open source software projects (Asklund and Bendix, 2002)	Compendex	IC1, IC3
9.	MQc	Achieving quality in open-source software (Aberdour, 2007)	Compendex	IC1, IC3
10.	MQc	Adapting the "staged model for software evolution" to free/libre/open source software (Capiluppi et al., 2007)	Compendex	IC1, IC3
11.	MQc	Collaboration, peer review and open source software (Johnson, 2006)	WebOfScience	IC1, IC3
12.	MQc	Commercial adoption of open source software: an empirical study (Glynn et al., 2005)	Compendex	IC1, IC2
13.	MQc	Open collaboration within corporations using software forges (Riehle et al., 2009)	IEEE	IC1
14.	MQc	Open knowledge management: lessons from the open source revolution (Awazu and Desouza, 2004)	WebOfScience	IC1, IC2, IC3
15.	MQc	Open source software—an evaluation (Fuggetta, 2003)	WebOfScience	IC1
16.	MQd	A soft-structured agile framework for larger scale systems development (Soundararajan and Arthur, 2009)	IEEE	IC1, IC3
17.	MQd	Adapting PROFES for use in an agile process: an industry experience report (Jedlitschka et al., 2005)	Compendex	IC1, IC3, IC4
18.	MQd	Addressing diverse needs through a balance of agile and plan-driven software development methodologies in the core software engineering course (Layman et al., 2008)	WebOfScience	IC1, IC3
19.	MQd	Agile development: Good process or bad attitude? (Turner, 2002)	WebOfScience	IC1, IC3
20.	MQd	Applying agile principles for distributed software development (Phalnikar et al., 2009)	IEEE	IC1, IC2
21.	MQd	Army simulation program balances agile and traditional methods with success (Surdu and Parsons, 2006)	Compendex	IC1, IC4
22.	MQd	Balancing agility and discipline: evaluating and integrating agile and plan-driven methods (Boehm and Turner, 2004)	Compendex	IC1, IC3

Table 12 (Continued)

#	Question	Title and reference	Library	Inclusion criteria
23.	MQd	Balancing the human and the engineering factors in software development (Mnkandla and Dwolatzky, 2004)	Compendex	IC1, IC3, IC4
24.	MQd	Can agile and traditional systems development approaches coexist? An ambidextrous view (Vinekar et al., 2006)	WebOfScience	IC1, IC2, IC3
25.	MQd	Certifying for CMM Level 2 and ISO9001 with XP@Scrum (Vriens, 2003)	IEEE	IC1, IC2, IC4
26.	MQd	Combining agile methods with stage-gate project management (Karlstrom and Runeson, 2005)	Compendex	IC1, IC4
27.	MQd	Designing an agile methodology for mobile software development: a hybrid method engineering approach (Rahimian and Ramsin, 2008)	IEEE	IC1, IC3
28.	MQd	Experiences using agile software development for a marketing simulation (Mills et al., 2006)	IEEE	IC1, IC4
29.	MQd	Formalizing agility, part 2: how an agile organization embraced the CMMI (Baker, 2006)	Compendex	IC1, IC2, IC4
30.	MQd	Future implementation and integration of Agile methods in software development and testing (Kee, 2006)	Compendex	IC1, IC2, IC4
31.	MQd	Management challenges to implementing agile processes in traditional development organizations (Boehm and Turner, 2005)	Compendex	IC1, IC2
32.	MQd	Mapping CMMI project management process areas to SCRUM practices (Marcal et al., 2007)	Compendex	IC1, IC3
33.	MQd	Mastering dual-shore development—the tools and materials approach adapted to agile offshoring (Kornstädt and Sauer, 2007)	Compendex	IC1, IC3
34.	MQd	Mature agile with a twist of CMMI (Jakobsen and Johnson, 2008)	Compendex	IC1, IC3, IC4
35.	MQd	Quantitative process improvement in XP using six sigma tools (Hashmi and Jongmoon Baik, 2008)	Compendex	IC1, IC3
36.	MQd	Reconciling agility and discipline in COTS selection processes (Navarrete et al., 2007)	Compendex	IC1, IC3
37.	MQd	Requirements engineering and agile software development (Paetsch et al., 2003)	IEEE	IC1, IC2, IC3
38.	MQd	Stretching agile to fit CMMI level 3—the story of creating MSF for CMMI process improvement at Microsoft corporation (Anderson, 2005)	Compendex	IC1, IC3
39.	MQd	The role of extreme programming in a plan-driven organization (Dahlberg et al., 2006)	Scopus	IC1, IC2, IC4
40.	MQd	Using risk to balance agile and plan-driven methods (Boehm and Turner, 2003a)	Compendex	IC1, IC3
41.	MQd	When software engineers met research scientists: A case study (Segal, 2005)	Compendex	IC1
42.	MQd	XP and the CMM (Reifer, 2003)	Compendex	IC1, IC2, IC3

References

- Aberdour, M., 2007. Achieving quality in open-source software. *IEEE Software* 24 (1), 58–64.
- Abrahamsson, P., Salo, O., Ronkainen, J., et al., 2002. Agile Software Development Methods: Review and Analysis. Technical Report. VTT Publications, p. 478.
- Anderson, D.J., 2005. Stretching agile to fit CMMI level 3—the story of creating MSF for CMMI® process improvement at Microsoft corporation. In: Agile Conference, Denver, USA, pp. 193–201.
- Anderson, D.J., 2010. Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.
- Angioni, M., Sanna, R., Soto, A., 2005. Defining a distributed agile methodology for an open source scenario. In: First International Conference on Open Source Systems, Genoa, Italy, pp. 209–214.
- Askund, U., Bendix, L., 2002. A study of configuration management in open source software projects. *IEE Proceedings-Software* 149, 40–46.
- Awazu, Y., Desouza, K.C., 2004. Open knowledge management: lessons from the open source revolution. *Journal of the American Society for Information Science and Technology* 55 (September (11)), 1016–1019.
- Babar, M.A., Zhang, H., 2009. Systematic literature reviews in software engineering: preliminary results from interviews with researchers. In: International Symposium on Empirical Software Engineering and Measurement (ESEM), Lake Buena Vista, FL, USA, pp. 346–355.
- Baker, S.W., 2006. Formalizing agility, part 2: how an agile organization embraced the CMMI. In: Agile Conference, pp. 1–8.
- Barnett, L., 2004. Applying open source processes in corporate development organizations. In: Forrester Research, pp. 1–15.
- Beck, K., 2004. Extreme Programming Explained: Embrace Change, 2nd ed. Addison-Wesley, Boston, MA, USA.
- Beck, K., Beedle, M., van Bennekum, A., et al., 2001. Manifesto for Agile Software Development, Available at: <http://agilemanifesto.org/> (last access: 15 December 2008).
- Bekkers, W., van de Weerd, I., Brinkkemper, S., et al., 2008. The influence of situational factors in software product management: an empirical study. In: International Workshop on Software Product Management (IWSPM), Barcelona, Catalonia, Spain, pp. 41–48.
- Biolchini, J., Mian, P.G., Natali, A.C., et al., 2005. Systematic Review in Software Engineering, Relatório Técnico ES-679, PESC-UFRJ. Available at: <http://www.cos.ufrj.br>.
- Boehm, B., 2002. Get ready for agile methods, with care. *IEEE Computer* 35 (1), 64–69.
- Boehm, B., Turner, R., 2003a. Using risk to balance agile and plan-driven methods. *Computer* 36 (6), 57–66.
- Boehm, B., Turner, R., 2004. Balancing agility and discipline: evaluating and integrating agile and plan-driven methods. In: International Conference on Software Engineering (ICSE), Scotland, UK, pp. 718–719.
- Boehm, B., Turner, R., 2005. Management challenges to implementing agile processes in traditional development organizations. *IEEE Software* 22 (5), 30–39.
- Boehm, B., Turner, R., 2003b. Balancing Agility and Discipline: A Guide for the Perplexed. Addison-Wesley, Boston, MA, USA.
- Brereton, P., Kitchenham, B.A., Budgen, D., et al., 2007. Lessons from applying the systematic literature review process within the software engineering domain. *Journal of Systems and Software* (JSS) 80 (4), 571–583.
- Capiluppi, A., González-Barahona, J.M., Herraiz, I., et al., 2007. Adapting the “staged model for software evolution” to free/libre/open source software. In: International Workshop on Principles of Software Evolution, Dubrovnik, Croatia, pp. 79–82.
- Capiluppi, A., Lago, P., Morisio, M., 2003. Characteristics of open source projects. In: Proceedings of the 17th European Conference on Software Maintenance and Reengineering, Washington, USA, pp. 1–11.
- Chrissis, M.B., Konrad, M., Shrum, S., 2006. CMMI: Guidelines for Process Integration and Product Improvement, 2nd ed. Addison-Wesley, Boston, MA, USA.
- Cockburn, A., 2001. Agile Software Development. Addison-Wesley, Boston, MA, USA.
- Cockburn, A., 2004. Crystal Clear: A Human-Powered Methodology for Small Teams, 1st ed. Addison-Wesley, Boston, MA, USA.
- Cubranic, D., Booth, K.S., 1999. Coordinating open-source software development. In: International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE), Stanford, USA, pp. 61–66.
- Cugola, G., Ghezzi, C., 1998. Software processes: a retrospective and a path to the future. *Software Process Improvement and Practice (SPIP) Journal* 4, 101–123.
- Dahlberg, H., Ruiz, F., Olsson, C., 2006. The role of extreme programming in a plan-driven organization. In: The Transfer and Diffusion of Information Technology for Organizational Resilience, pp. 291–312.
- Deck, M., 2001. Managing process diversity while improving your practices. *IEEE Software* 18 (May (3)), 21–27.
- DeMarco, T., Lister, T., 1999. Peopleware: Productive Projects and Teams, 2nd ed. Dorset House, New York, USA.
- Deshpande, A., Riehle, D., 2008. Continuous integration in open source software development. In: Open Source Development, Communities and Quality, 1st ed. Springer, Boston, MA, USA, pp. 273–280 (chapter 275).
- DiBona, C., Ockman, S., Stone, M., et al., 1999. Open Sources: Voices from the Open Source Revolution, 1st ed. O'Reilly Media, Inc.

- Dyba, T., Dingsoyr, T., 2008. Empirical studies of agile software development: a systematic review. *Information and Software Technology* 50 (August (9/10)), 833–859.
- Ebert, C., 2007. Open source drives innovation. *IEEE Software* 24 (3), 105–109.
- Feller, J., Fitzgerald, B., 2001. *Understanding Open Source Software Development*. Addison-Wesley, Boston, MA, USA.
- Florac, W.A., Carleton, A.D., 1999. *Measuring the Software Process: Statistical Process Control for Software Process Improvement*. Addison-Wesley Professional, Boston, MA, USA.
- FSF, 2008. The Free Software Definition, Available at: <http://www.gnu.org/philosophy/free-sw.html> (last access: 31 May 2008).
- Fuggetta, A., 2003. Open source software—an evaluation. *Journal of Systems and Software (JSS)* 66 (1), 77–90.
- Fuggetta, A., 2000. Software process: a roadmap. In: *Proceedings of the Conference on the Future of Software Engineering*, Limerick, Ireland, pp. 25–34.
- Ginsberg, M., Quinn, L., 1995. *Process Tailoring and the Software Capability Maturity Model CMU/SEI-94-TR-024*, SEI-CMU. Available at: <http://www.sei.cmu.edu/publications/documents/94.reports/94.tr.024.html>.
- Glass, R.L., 2001. Agile versus traditional: make love, not war! *Cutter IT Journal* 14 (12), 12–18.
- Glazer, H., Dalton, J., Anderson, D., et al., 2008. CMMI or Agile: Why Not Embrace Both! SEI-CMU, Available at: <http://www.sei.cmu.edu/publications/documents/08.reports/08tn003.html>.
- Glynn, E., Fitzgerald, B., Exton, C., 2005. Commercial adoption of open source software: an empirical study. In: *International Symposium on Empirical Software Engineering*, Los Alamitos, CA, USA, p. 10.
- Goth, G., 2007. Sprinting toward open source development. *IEEE Software* 24 (1), 88–91.
- Hashmi, S.I., Jongmoon Baik, 2008. Quantitative process improvement in XP using six sigma tools. In: *International Conference on Computer and Information Science (ICIS)*, Portland, OR, USA, pp. 519–524.
- Huang, M., Yang, L., Yang, Y., 2006. A development process for building OSS-based applications. In: *Unifying the Software Process Spectrum*, pp. 122–135.
- Humphrey, W.S., 1989. *Managing the Software Process*. Addison-Wesley, Boston, MA, USA.
- ISO/IEC, 1995. *ISO/IEC 12207: Information Technology—Software Life Cycle Processes*. ISO/IEC, Geneva.
- Jakobsen, C.R., Johnson, K.A., 2008. Mature agile with a twist of CMMI. In: *Proceedings of the Agile 2008*, Girona, Catalonia, Spain, pp. 212–217.
- Jaufman, O., Munch, J., 2005. Acquisition of a project-specific process. In: *Product Focused Software Process Improvement*. Springer Verlag, Berlin/Heidelberg, pp. 328–342.
- Jedlitschka, A., Hamann, D., Göhlert, T., et al., 2005. Adapting PROFES for use in an agile process: an industry experience report. In: *International Conference on Product Focused Software Process Improvement (PROFES)*, Oulu, Finland, pp. 502–516.
- Johnson, J.P., 2006. Collaboration, peer review and open source software. *Information Economics and Policy* 18 (November (4)), 477–497.
- Karlstrom, D., Runeson, P., 2005. Combining agile methods with stage-gate project management. *IEEE Software* 22 (3), 43–49.
- Kee, W.H., 2006. Future implementation and integration of agile methods in software development and testing. In: *International Conference on Innovations in Information Technology*, Piscataway, United States, pp. 1–5.
- Kitchenham, B., 2004. *Procedures for Performing Systematic Reviews*, Technical Report TR/SE-0401. Software Engineering Group, Department of Computer Science, Keele University. Available at: <http://www.idi.ntnu.no/emner/empse/papers/kitchenham.2004.pdf>.
- Kitchenham, B., Brereton, P., Turner, M., et al., 2009a. The impact of limited search procedures for systematic literature reviews—a participant-observer case study. In: *3rd International Symposium on Empirical Software Engineering and Measurement (ESEM)*, Lake Buena Vista, FL, USA, pp. 336–345.
- Kitchenham, B.A., Mendes, E., Travassos, G.H., 2007. Cross versus within-company cost estimation studies: a systematic review. *IEEE Transactions on Software Engineering* 33 (5), 316–329.
- Kitchenham, B., Charters, S., 2007. Guidelines for performing systematic literature reviews in software engineering, Technical Report EBSE 2007-001, Keele University and Durham University Joint Report. Available at: <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.117.471>.
- Kitchenham, B., Pearl Brereton, O., Budgen, D., et al., 2009b. Systematic literature reviews in software engineering—a systematic literature review. *Information and Software Technology* 51 (1), 7–15.
- Koch, S., 2004. Agile principles and open source software development: a theoretical and empirical discussion. In: *Extreme Programming and Agile Processes in Software Engineering*, 1st ed. Springer-Verlag, Heidelberg, pp. 85–93 (chapter 3092).
- Kornstädt, A., Sauer, J., 2007. Mastering dual-shore development: the tools and materials approach adapted to agile offshoring. In: *International Conference on Software Engineering Approaches for Offshore and Outsourced Development*, Zurich, Switzerland, pp. 83–95.
- Layman, L., Williams, L., Slaten, K., et al., 2008. Addressing diverse needs through a balance of agile and plan-driven software development methodologies in the core software engineering course. *International Journal of Engineering Education* 24 (4), 659–670.
- Lindvall, M., Rus, I., 2000. Process diversity in software development. *IEEE Software* 17 (August (4)), 14–18.
- MacDonell, S., Shepperd, M., Kitchenham, B., et al., 2010. How reliable are systematic reviews in empirical software engineering? *IEEE Transactions on Software Engineering* 36 (September), 676–687.
- Magdaleno, A.M., 2010a. Balancing collaboration and discipline in software development processes. In: *Doctoral Symposium of International Conference on Software Engineering (ICSE)*, Cape Town, South Africa, pp. 331–332.
- Magdaleno, A.M., 2010b. An optimization-based approach to software development process tailoring. In: *PhD Track—International Symposium on Search Based Software Engineering (SSBSE)*, Benevento, Italy, pp. 40–43.
- Magdaleno, A.M., Werner, C.M.L., Araujo, R.M., 2010. Analyzing collaboration in software development processes through social networks. In: *International Symposium on Leveraging Applications of Formal Methods. Verification and Validation (IsoLA)*, Heraklion, Crete, Greece, pp. 435–446.
- Marcal, A.S.C., Soares, F.S.F., Belchior, A.D., 2007. Mapping CMMI project management process areas to SCRUM practices. In: *Proceedings of the 31st IEEE Software Engineering Workshop*, pp. 13–22.
- Miller, G.G., 2001. The characteristics of agile software processes. In: *Proceedings of the 39th International Conference and Exhibition on Technology of Object-Oriented Languages and Systems (TOOLS)*, Santa Barbara, CA, USA, pp. 1–3.
- Mills, D., Sherrell, L., Boydston, J., et al., 2006. Experiences using agile software development for a marketing simulation. In: *Proceedings of the IEEE Southeast Conference*, pp. 285–290.
- Mnkandla, E., Dwolatzky, B., 2004. Balancing the human and the engineering factors in software development. In: *7th AFRICON Conference in Africa*, vol. 2, pp. 1207–1210.
- Moreira, M.E., 2009. *Adapting Configuration Management for Agile Teams: Balancing Sustainability and Speed*. Wiley.
- Navarrete, F., Botella, P., Franch, X., 2007. Reconciling agility and discipline in COTS selection processes. In: *Sixth International IEEE Conference on Commercial-off-the-Shelf (COTS)-Based Software Systems (ICCBSS)*, Banff, Alberta, Canada, pp. 103–113.
- Paetsch, F., Eberlein, A., Maurer, F., 2003. Requirements engineering and agile software development. In: *Proceedings of the Twelfth International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*, p. 308.
- Pai, M., McCulloch, M., Gorman, J.D., et al., 2004. Systematic reviews and meta-analyses: an illustrated, step-by-step guide. *The National Medical Journal of India* 17 (2), 86–95.
- Park, S., Na, H., Park, S., et al., 2006. A semi-automated filtering technique for software process tailoring using neural network. *Expert Systems with Applications* 30 (February (2)), 179–189.
- Pedreira, O., Piatini, M., Luaces, M.R., et al., 2007. A systematic review of software process tailoring. *SIGSOFT Software Engineering Notes* 32 (3), 1–6.
- Phalnikar, R., Deshpande, V.S., Joshi, S.D., 2009. Applying agile principles for distributed software development. In: *International Conference on Advanced Computer Control*, Los Alamitos, CA, USA, pp. 535–539.
- Pressman, R.S., 2001. *Software Engineering: A Practitioner's Approach*, 5th ed. McGraw-Hill.
- Rahimian, V., Ramsin, R., 2008. Designing an agile methodology for mobile software development: a hybrid method engineering approach. In: *International Conference on Research Challenges in Information Science (RCIS)*, Marrakech, Morocco, pp. 337–342.
- Raman, S., 2000. It is software process, stupid: next millennium software quality key. *IEEE Aerospace and Electronic Systems Magazine* 15 (6), 33–37.
- Ramesh, B., Cao, L., Mohan, K., et al., 2006. Can distributed software development be agile? *Communications of ACM* 49 (10), 41–46.
- Raymond, E.S., 2001. *The Cathedral & the Bazaar*, revised & expanded ed. O'Reilly Media.
- Reifer, D.J., 2003. XP and the CMM. *IEEE Software* 20 (3), 14–15.
- Riehle, D., Ellenberger, J., Menahem, T., et al., 2009. Open collaboration within corporations using software forges. *IEEE Software* 26 (2), 52–58.
- Santana, C., Gusmão, C., Soares, L., et al., 2009. Agile software development and CMMI: what we do not know about dancing with elephants. In: *Agile Processes in Software Engineering and Extreme Programming*. Springer, Heidelberg, pp. 124–129 (chapter 31).
- Schwaber, K., 2004. *Agile Project Management with Scrum*. Microsoft Press, Washington, DC, USA.
- Segal, J., 2005. When software engineers met research scientists: a case study. *Empirical Software Engineering* 10 (4), 517–536.
- Sharma, S., Sugumaran, V., Rajagopalan, B., 2002. A framework for creating hybrid-open source software communities. *Information Systems Journal* 12 (January (1)), 7–25.
- Siebel, N.T., Cook, S., Satpathy, M., et al., 2003. Latitudinal and longitudinal process diversity. *Journal of Software Maintenance: Research and Practice* 15 (January (1)), 9–25.
- Simmons, G., Dillon, T., 2003. A critical comparison of agile methods and open source development through a case study. In: *International Conference on Software Engineering and their Applications*, Paris, France.
- Sommerville, I., 2004. *Software Engineering*, 7th ed. Addison Wesley.
- Soundararajan, S., Arthur, J.D., 2009. A soft-structured agile framework for larger scale systems development. In: *16th Annual IEEE International Conference and Workshop on the Engineering of Computer Based Systems (ECBS)*, pp. 187–195.
- Surdu, J., Parsons, D.J., 2006. Army simulation program balances agile and traditional methods with success. In: *STSC CrossTalk*, Available at: <http://www.stsc.hill.af.mil/crosstalk/2006/04/0604surduparsons.html> (last access: 28 October 2010).

- Theunissen, M., Kourie, D., Boake, A., 2008. Corporate-, agile- and open source software development: a witch's brew or an elixir of life. In: *Balancing Agility and Formalism in Software Engineering: Second IFIP TC 2 Central and East European Conference on Software Engineering Techniques (CEE-SET)*. Springer-Verlag, pp. 84–95.
- Travassos, G.H., dos Santos, P.S.M., Neto, P.G.M., et al., 2008. An environment to support large scale experimentation in software engineering. In: *International Conference on the Engineering of Complex Computer Systems (ICECCS)*, Belfast, United Kingdom, pp. 193–202.
- Tsirakidis, P., Kobler, F., Krcmar, H., 2009. Identification of success and failure factors of two agile software development teams in an open source organization. In: *Proceedings of the 2009 Fourth IEEE International Conference on Global Software Engineering*, Limerick, Ireland, pp. 295–296.
- Turner, R., 2002. Agile development: good process or bad attitude? In: *Proceedings of the 4th International Conference on Product Focused Software Process Improvement*, pp. 134–144.
- Turnu, I., Melis, M., Cau, A., et al., 2006. Modeling and simulation of open source development using an agile practice. *Journal of Systems Architecture (JSA)* 52 (11), 610–618.
- Vinekar, V., Slinkman, C.W., Nerur, S., 2006. Can agile and traditional systems development approaches coexist? An ambidextrous view. *Information Systems Management (ISM)* 23 (3), 31–42.
- Vreede, G.-J., Briggs, R.O., 2005. Collaboration engineering: designing repeatable processes for high-value collaborative tasks. In: *Annual Hawaii International Conference on System Sciences (HICSS)*, Hilton Waikoloa Village, HI, USA, pp. 1–10.
- Vriens, C., 2003. Certifying for CMM level 2 and ISO9001 with XP@Scrum. In: *Proceedings of the Conference on Agile Development*, p. 120.
- Warsta, J., Abrahamsson, P., 2003. Is open source software development essentially an agile method? In: *Proceedings of the Workshop on Open Source Software Development*, Portland, OR, USA, pp. 143–147.
- Xu, P., Ramesh, B., 2008. Using process tailoring to manage software development challenges. *IT Professional* 10 (4), 39–45.
- Andréa Magalhães Magdaleno** is a D.Sc. student at the Computer Science Department of COPPE/UFRJ, Brazil. She obtained her M.Sc. degree in computer science at UFRJ in 2007. She is the author of publications in Brazilian and international conferences and journals. She coordinates consulting projects at companies in the field of business process modeling and requirements engineering. Her areas of interest are: software engineering, collaboration, and business process management.
- Renata Mendes de Araujo** obtained her D.Sc. in computer science from COPPE/UFRJ (2000). She is an associate professor at UNIRIO (Rio de Janeiro State Federal University), Brazil, and a CNPq researcher. Her experience and research work focus on information systems, software engineering, software development process improvement, business process management, computer-supported cooperative work and e-democracy. She coordinates consulting projects in the field of software development and business process modeling.
- Cláudia Maria Lima Werner** holds a D.Sc. in computer science from COPPE/UFRJ (1992). She is an associate professor at COPPE/UFRJ, leads the software engineering group, and is currently responsible for information and communication technology at UFRJ. She is also a CNPq and FAPERJ researcher. She has published several papers and served on program committees of both Brazilian and international conferences and journals. She has extensive experience in computer science with an emphasis on software engineering in the following research areas: software reuse, component-based development, and software product line.